

The hidden infra behind "For You" pages

Attila Toth · Senior Developer Advocate, ScyllaDB

Attila Toth

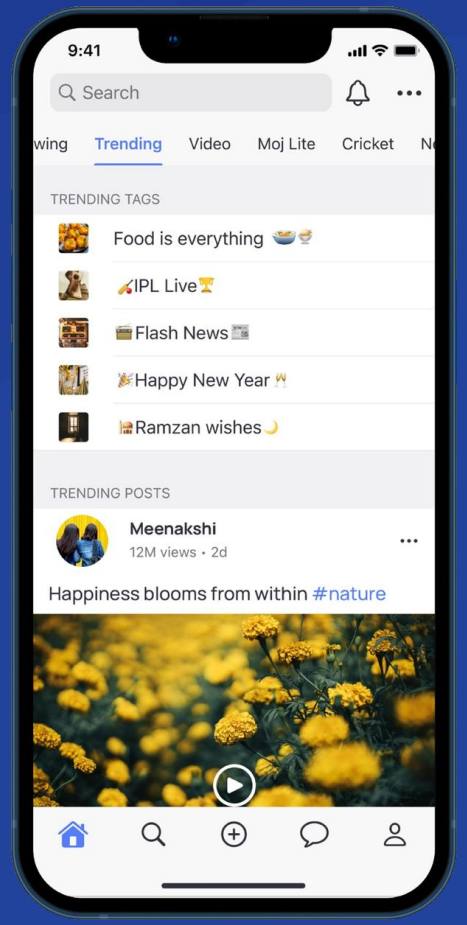
- Developer advocate at ScyllaDB
- Working as a software engineer + dev advocate in the data space for 9 years
- Lives in Budapest, Hungary

Open source contributions

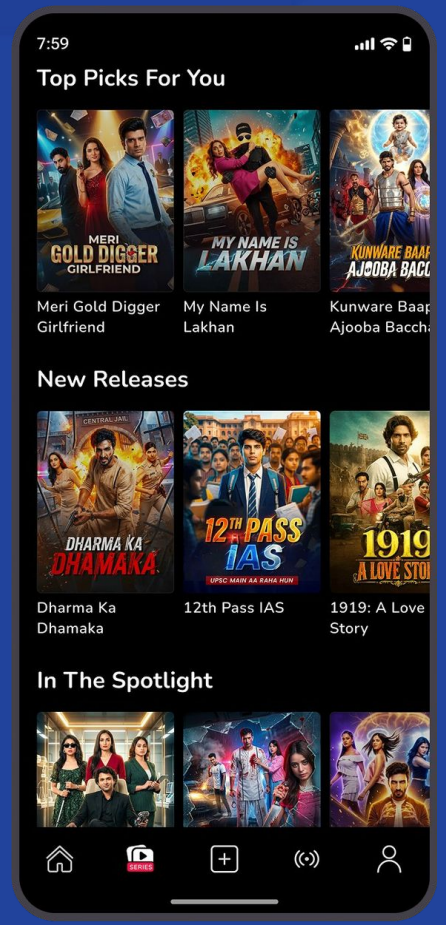




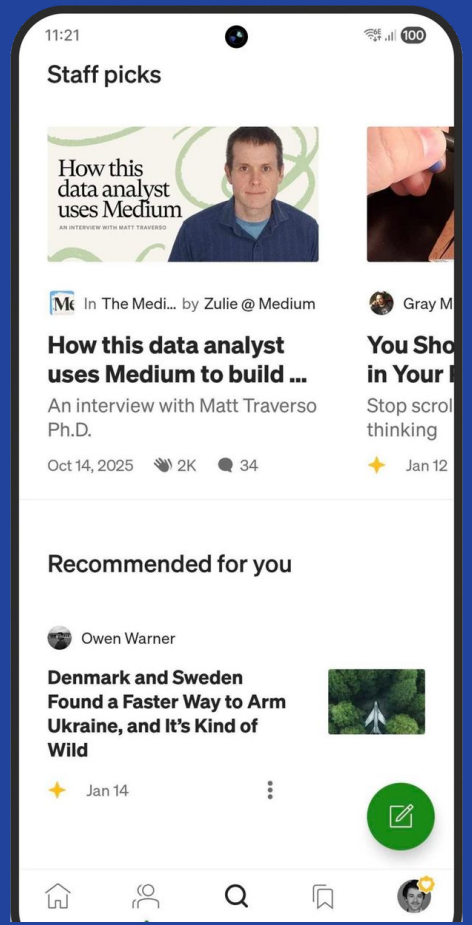
The For You feed



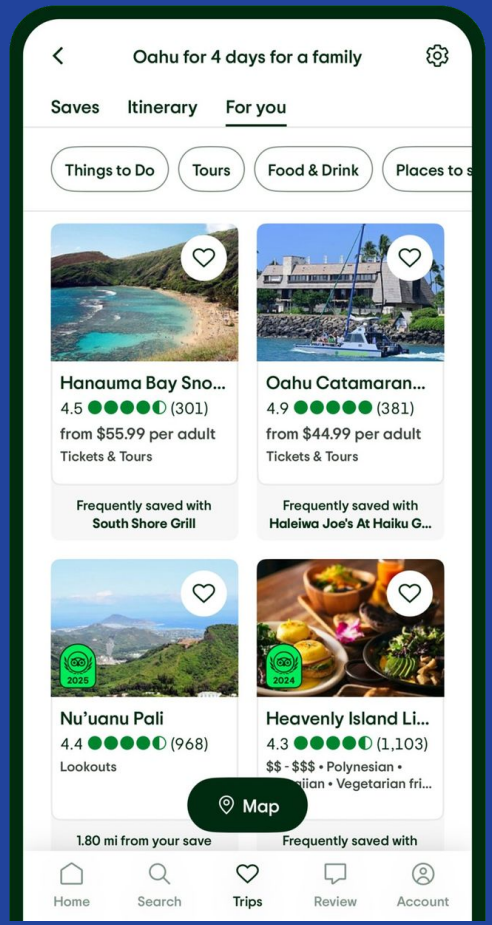
ShareChat



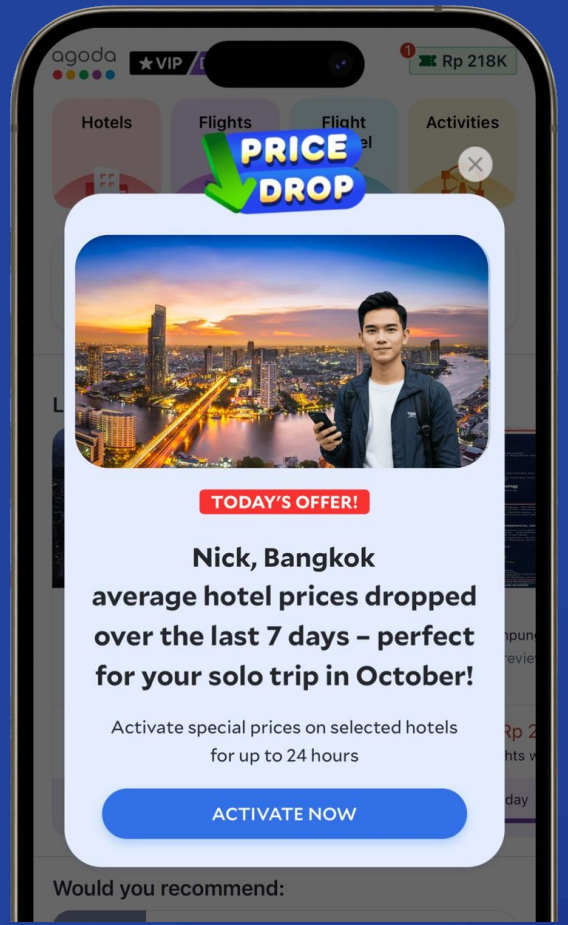
Moj



Medium



Tripadvisor



Agoda

SCYLLA

Every swipe asks for the next ranked items



Moj



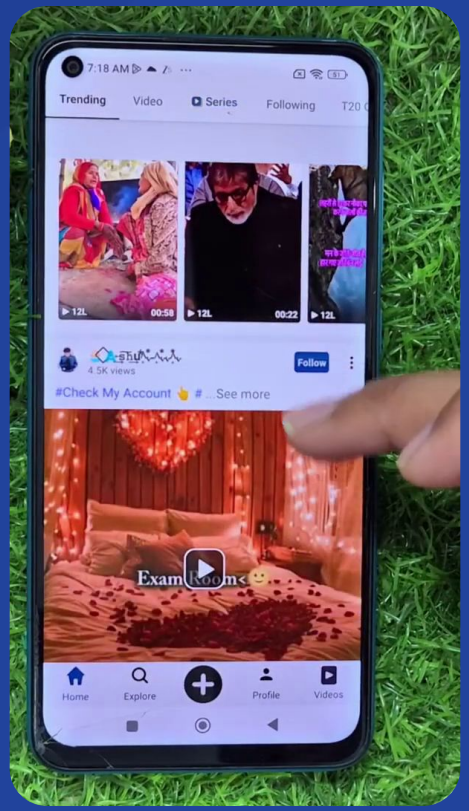
ShareChat

Footage: youtube.com/watch?v=rarFJ_4WGh0 · youtube.com/watch?v=2KzQk9ZN7oU

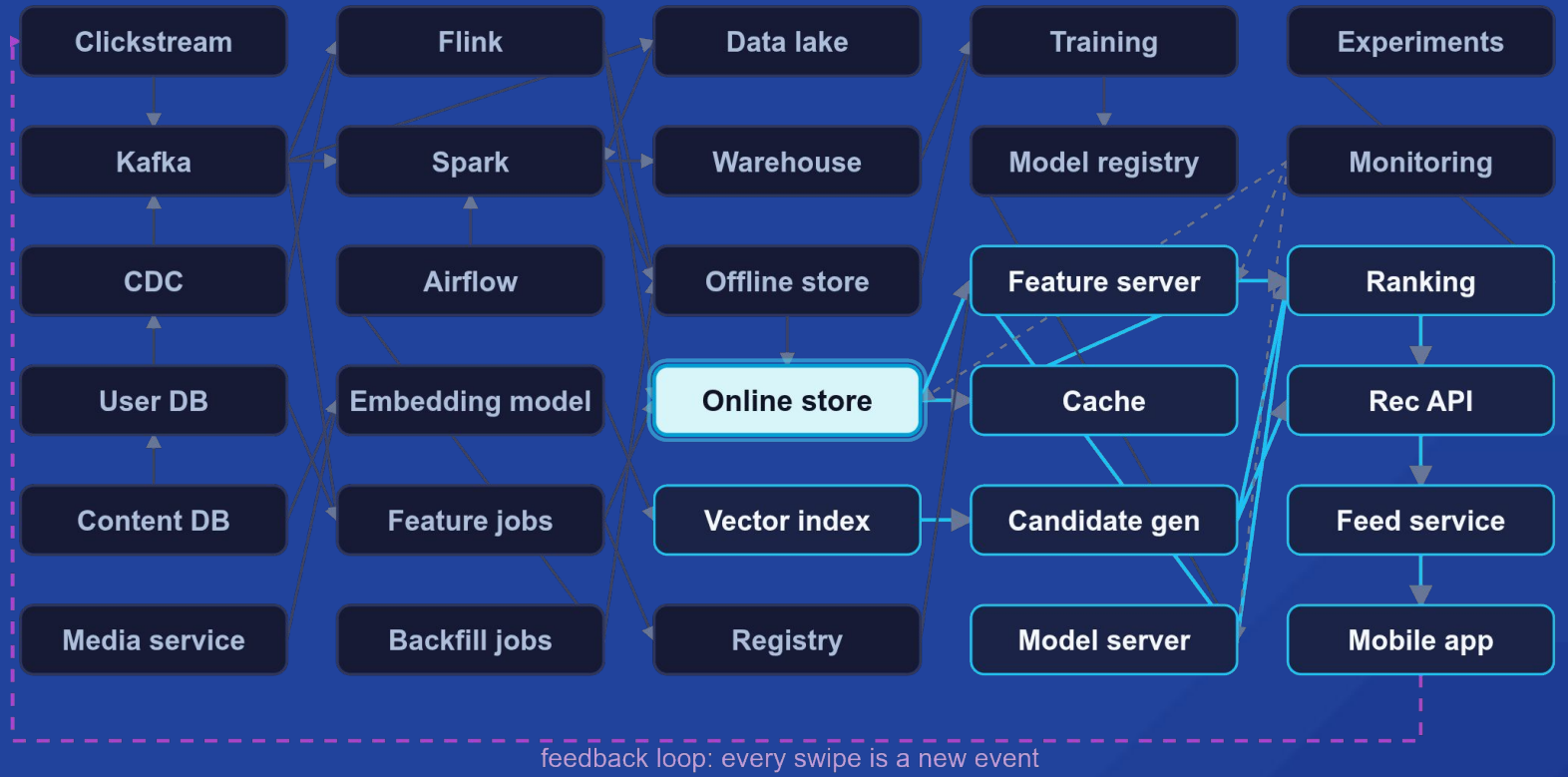
SCYLLA

Behind every swipe

WHAT THE USER SEES



WHAT RUNS UNDER THE HOOD



Footage: youtube.com/watch?v=2KzQk9ZN7oU

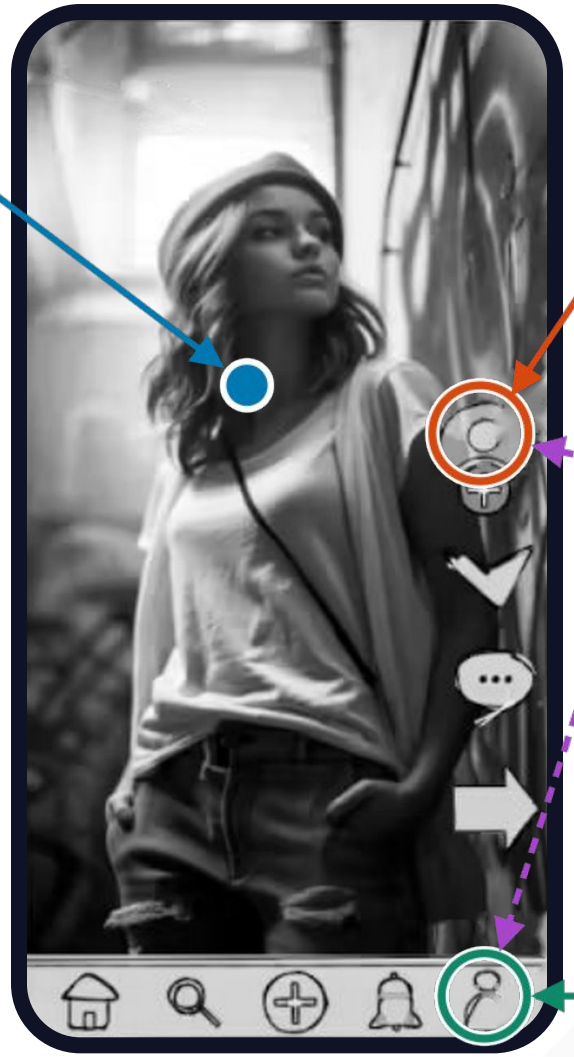


Feature store basics

What is a feature?

Post

```
post_likes_10m
post_shares_7d
post_topic
```



Creator

```
creator_comments_30d
creator_views_1h
creator_state
```

Cross: user × creator

```
user_creator_likes_30d
user_creator_dwell_time_7d
```

User

```
user_time_spent_7d
user_video_plays_30m
user_dob
```

READING A NAME

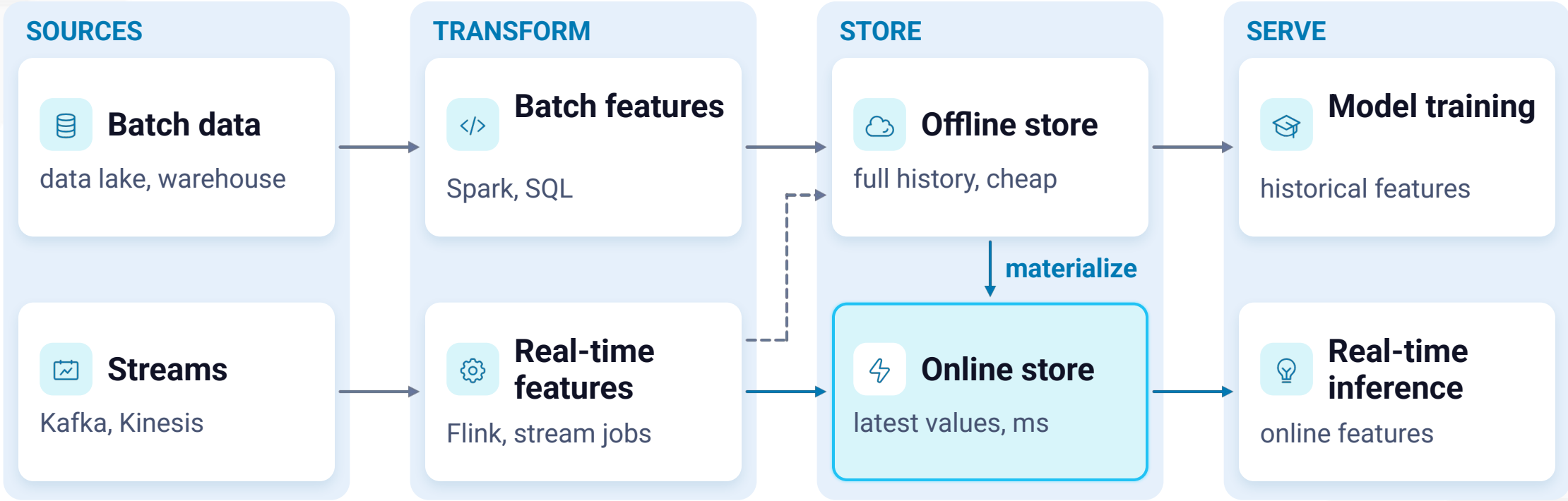
`post`_`likes_10m`

entity signal window

Short windows need fresh data.

SCYLLA

Feature store = offline + online + registry

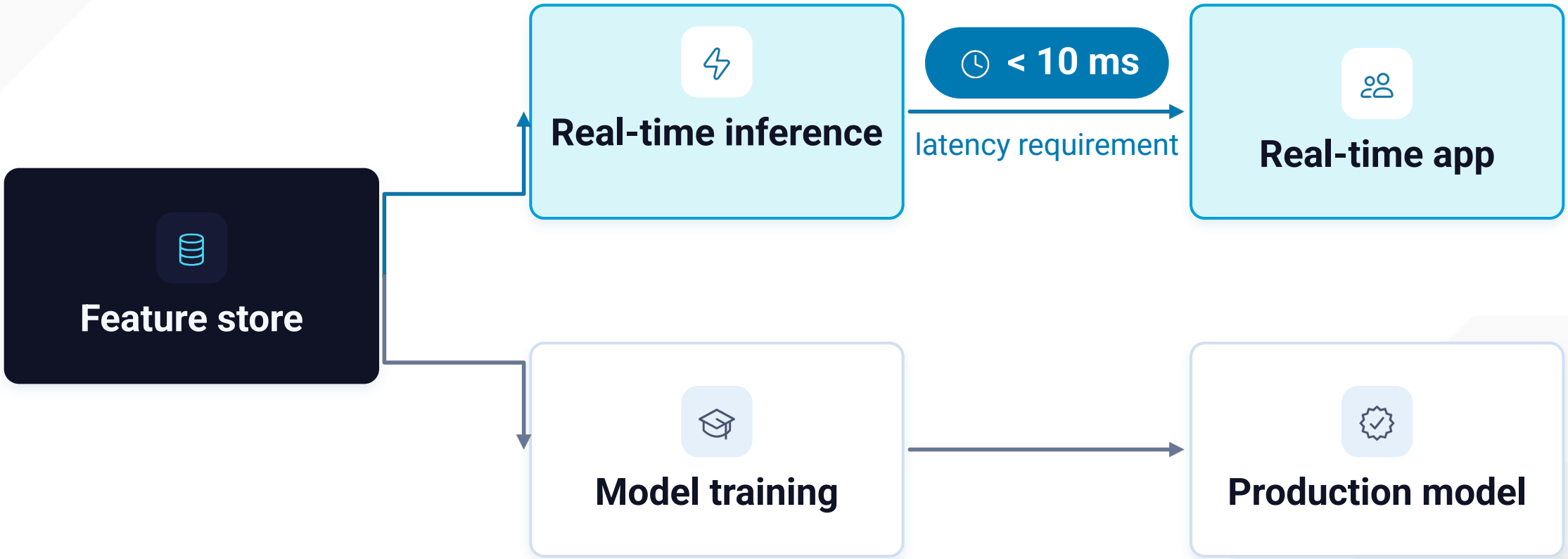


SCYLLA

Why use a feature store

Reuse	Build a feature once. Every model uses the same one.
Share	One catalog of features across teams.
Self-serve	Data scientists create and read features without waiting for engineering.
One pipeline	Training and online inference use the same feature definitions. No training-serving skew.
One system	Different ML workloads run on one platform.

Training vs. inference workloads

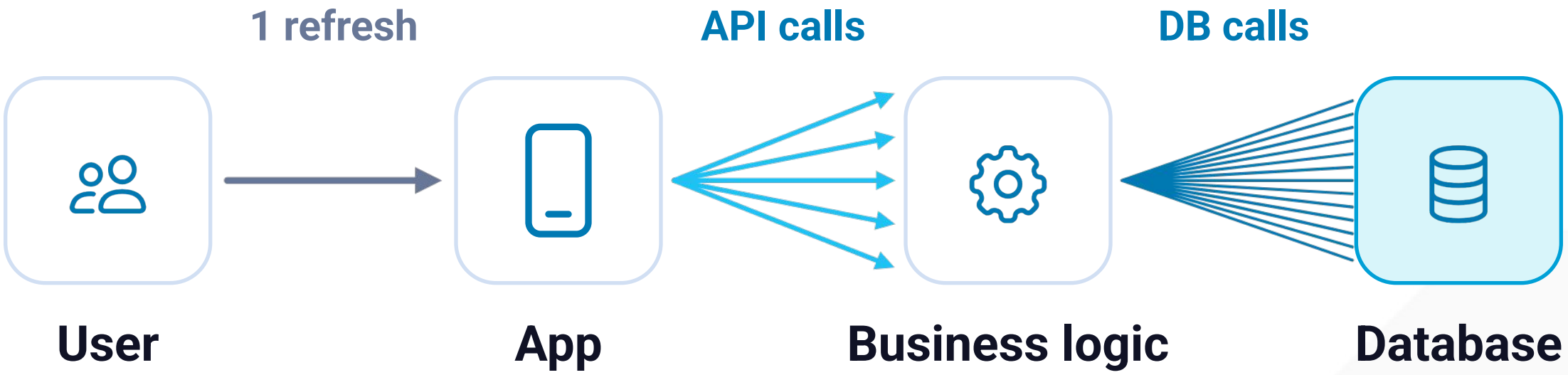


SCYLLA

Training vs. inference workloads

	Model training	Real-time inference
Access	Batch jobs	**Request stream** constant, every page view
Per request	Billions of rows	**1 feature vector** one entity, latest values
Latency	Doesn't matter	**< 10 ms** milliseconds, often under 10

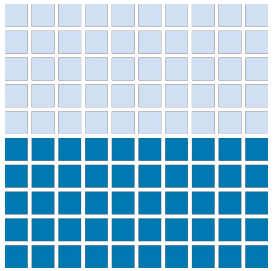
Why is latency important?



One refresh fans out to many API calls and many more DB calls.

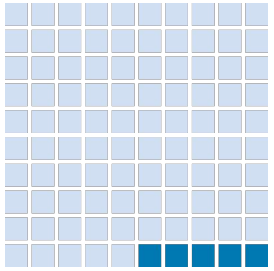
One slow lookup slows the whole page

p50



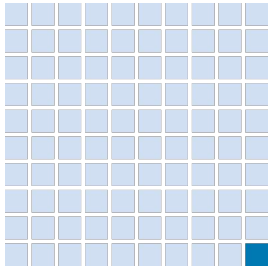
1 in 2 is slower
the median

p95



1 in 20 is slower
5 out of 100

p99



1 in 100 is slower
the tail

A PAGE WAITS FOR 100 LOOKUPS

One lookup is fast

$$0.99$$

All 100 are fast

$$0.99^{100} \approx 0.37$$

At least one is slow

$$1 - 0.37 \approx 0.63$$

37% fast	63% wait on a slow one

Source: Dean & Barroso, The Tail at Scale, CACM 2013 (same 100-server, 63% example)

SCYLLA


low latency
cheap
storage

Most feature data stays offline

- Offline: full history for training and backfills
- Object storage: S3, GCS + Parquet, Iceberg, Delta
- Cost per TB matters more than latency
- Online store keeps only what serving needs, with TTL

Example (Tripadvisor)

Offline store (data warehouse)



125 TB

Online store (ScyllaDB)



9 TB

Source: youtube.com/watch?v=fttwnNaIUYI&t=702s

SCYLLA



Data modeling

Part 3

Two ways to design a data model

RELATIONAL · ENTITY-FIRST · DATA → MODEL → APP

01
Entities
Model the domain: entities and relations.

→

02
Normalized tables
One table per entity. No duplicate data.

→

03
Queries
Write the queries. JOINs combine the tables.

NoSQL · QUERY-FIRST · APP → DATA → MODEL

01
Data
List the entities and the data you need.

→

02
Queries
Map the app workflow to each query.

→

03
Tables
One table per query. One partition per read.

Sources: scylladb.com/glossary/nosql-data-modeling/ · ScyllaDB University S201: Data Modeling

SCYLLA

Design the table for the query

Table: list features

```
CREATE TABLE list_items (
  feature_key text,
  entity_id   text,
  item_key    text, -- timestamp + md5(value)
  value       text,
  PRIMARY KEY ((feature_key, entity_id), item_key)
) WITH CLUSTERING ORDER BY (item_key DESC);
```

Query: get list items

```
SELECT item_key, value
FROM list_items
WHERE feature_key = ?
  AND entity_id = ?
LIMIT ?;
```

1 query One list is one partition. Newest items come first, so LIMIT returns the latest N.

Source: scylladb.com/2026/08/11/medium-real-time-recommendations/ · Column types simplified.

One request = one partition

```
-- all list items for all list features
CREATE TABLE list_items (
  feature_key TEXT,
  entity_id TEXT,
  item_key TEXT,
  value BLOB,
  PRIMARY KEY ((feature_key, entity_id), item_key)
) WITH CLUSTERING ORDER BY (item_key DESC)
  AND DEFAULT_TIME_TO_LIVE = $defaultTTL;
```

Partition key

(feature_key, entity_id): one list = one partition.

Clustering key

timestamp + hash(value): newest first, no clashes.

Value

The serialized item (a blob).

Source: scylladb.com/2026/08/11/medium-real-time-recommendations/

Vector embeddings

```
CREATE TABLE "feast"."demo_documents" (
  entity_key   TEXT,
  feature_name TEXT,
  value        BLOB,
                                ,
  event_ts     TIMESTAMP,
  created_ts   TIMESTAMP,
  PRIMARY KEY ((entity_key), feature_name)
);

CREATE CUSTOM INDEX demo_documents_vec_idx
  ON "feast"."demo_documents" (vector_value)
  USING 'vector_index'
  WITH OPTIONS = {'similarity_function': 'COSINE'};
```

entity_key	feature_name	value	vector_value
doc_42	title	0x0a…	null
doc_42	category	0x0a…	null
doc_42	embedding	0x0a…	[0.12, …]

The embedding is one more feature row, in the same table as the other features.

The vector index makes vector_value searchable with ANN OF.

Source: github.com/feast-dev/feast · sdk/python/feast/infra/online_stores/scylladb_online_store/scylladb.py · Sample rows illustrative.

SCYLLA

Time-To-Live

```
BEGIN BATCH
  -- for each $item in $items:
  INSERT INTO list_items(
    feature_key, entity_id, item_key, value)
  VALUES (
    '$entityType#$featureName|$featureVersion',
    $entityID,
    ${buildItemKey(item)},
    ${item.Value})
  USING TTL ${item.Timestamp + ttl - now};
APPLY BATCH;
```

Logged batch: atomic insert. **Per-row TTL:** storage plateaus.

```
SELECT value, item_key
FROM list_items
WHERE feature_key = '...'
  AND entity_id = $entityID
  AND item_key >= ${buildItemKey(minTimestamp)}
ORDER BY item_key DESC
LIMIT $limit;
```

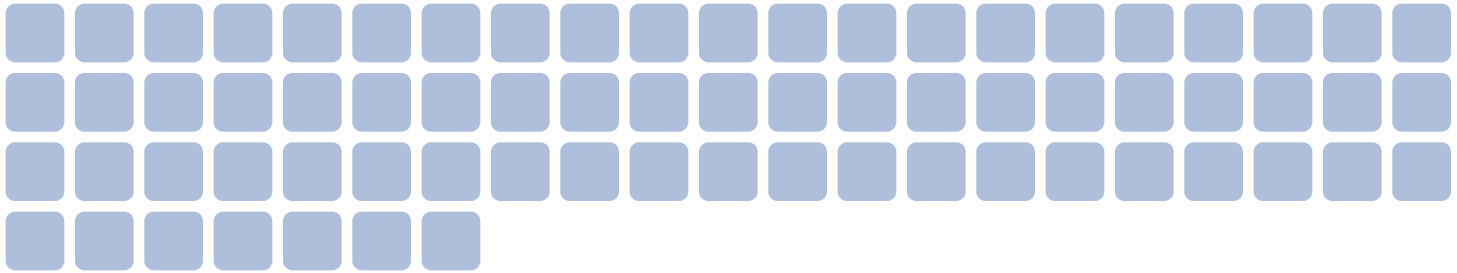
Single partition · range on the clustering key · no sort at read time.

Source: scylladb.com/2026/08/11/medium-real-time-recommendations/

Pre-aggregate into time buckets

A tile is one row that counts events in one time bucket, e.g. likes on a post per minute. Flink writes the tiles. The feature service sums the tiles in each window: 1h, 1d, 7d, 30d.

Tiles: 1m, 30m, 1d



~70
tiles per feature per query

+ 5m, 3h, 5d



23
tiles per feature per query

Rows/s needed

~2B
rows/s

protobuf rows →

200M
rows/s

+ 5m, 3h, 5d tiles →

~73M
rows/s

Protobuf rows: one binary row holds all features of a tile, not one row per feature.

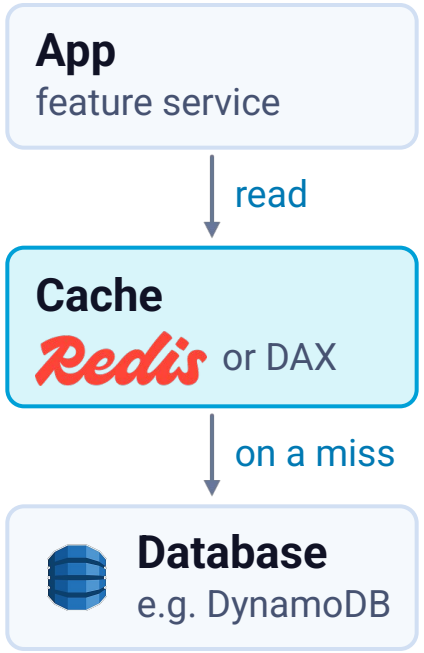
Source: scylladb.com/2024/08/27/how-sharechat-scaled-their-ml-feature-store/

SCYLLA

Cache?

External cache

A second system in front of the database



Costs

Extra hop and one more cluster to run.

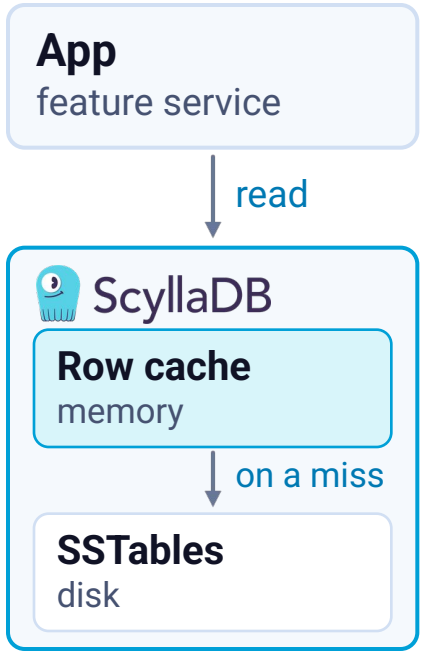
Invalidation: your code keeps cache and DB in sync.

Stale reads when the two drift apart.

Cold cache: after a failure, all misses hit the DB at once.

Embedded cache

Built into each database node



How it works

Row cache in every node. Reads fill it by default.

Same CPU core as the data it caches.

Writes: memtable flushes merge into it.

Same consistency rules as the database.

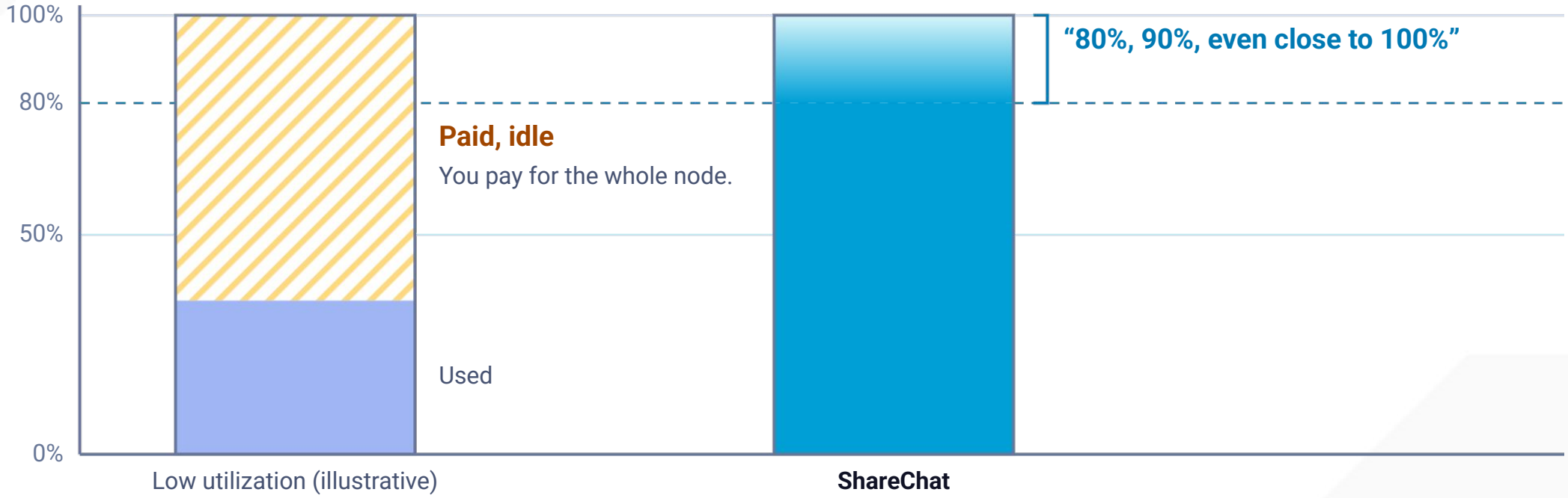
Sources: scylladb.com/2024/05/29/eliminating-external-database-caches/ · scylladb.com/2024/01/08/inside-scylladbs-internal-cache/

SCYLLA



Cost levers

Resource utilization

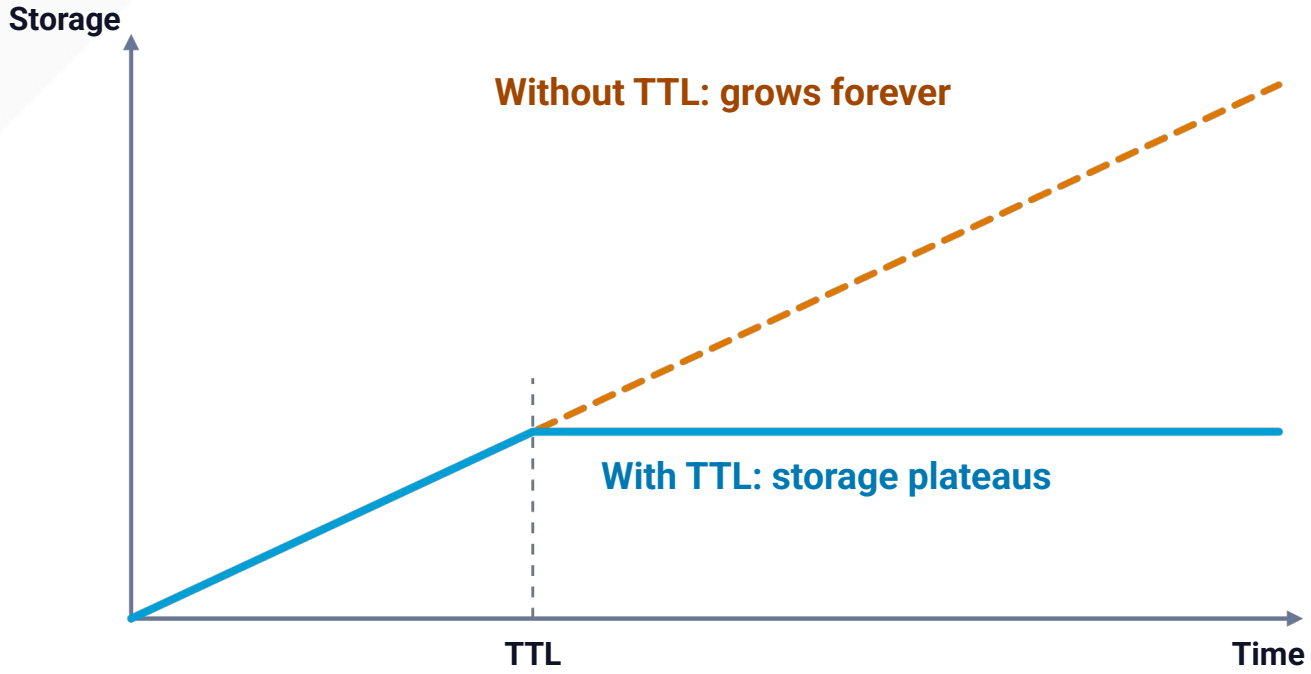


30+ clusters ShareChat runs its ScyllaDB clusters at 80–100% utilization.

Source: scylladb.com/2026/03/04/optimizing-a-fast-feature-store-for-costs-lessons-learned/

SCYLLA

Expiring data



One row, one TTL

```
INSERT INTO items … USING TTL ?
```

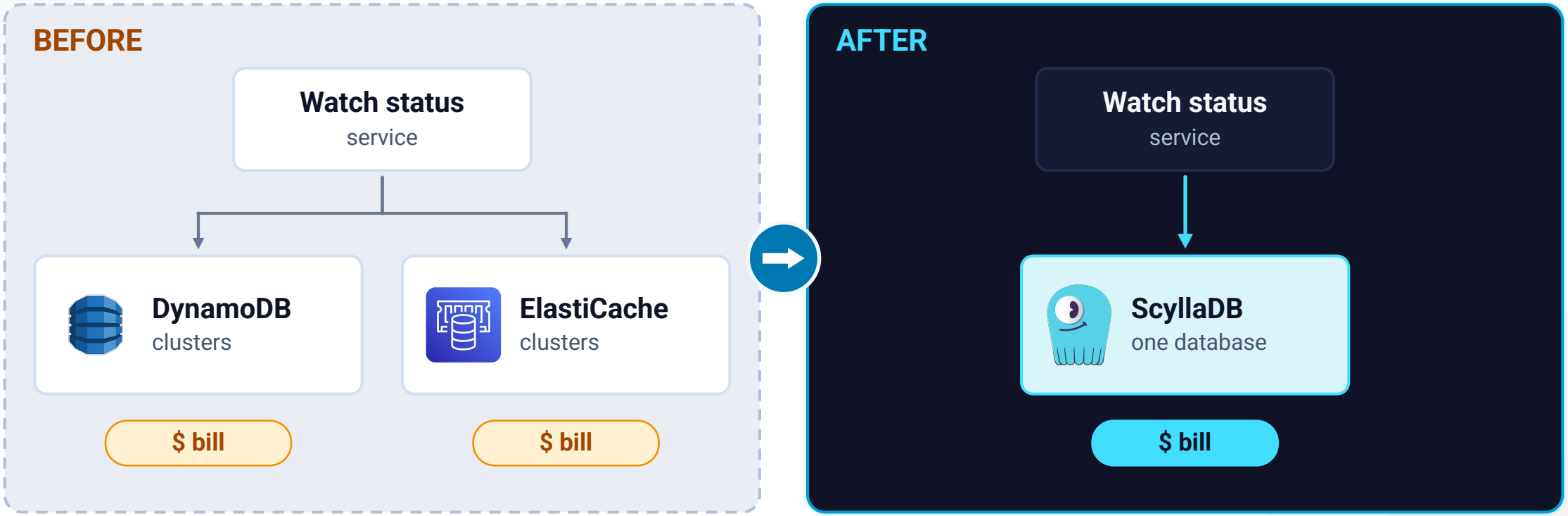


Expired rows count as deleted. No cleanup job.

6 months TTL of Medium's reading-history feature. Every row expires on its own, so storage plateaus. Retired features drain away.

Source: scylladb.com/2026/08/11/medium-real-time-recommendations/

Simplifying your stack



Source: youtube.com/watch?v=BsC0TswPzxg&t=259s

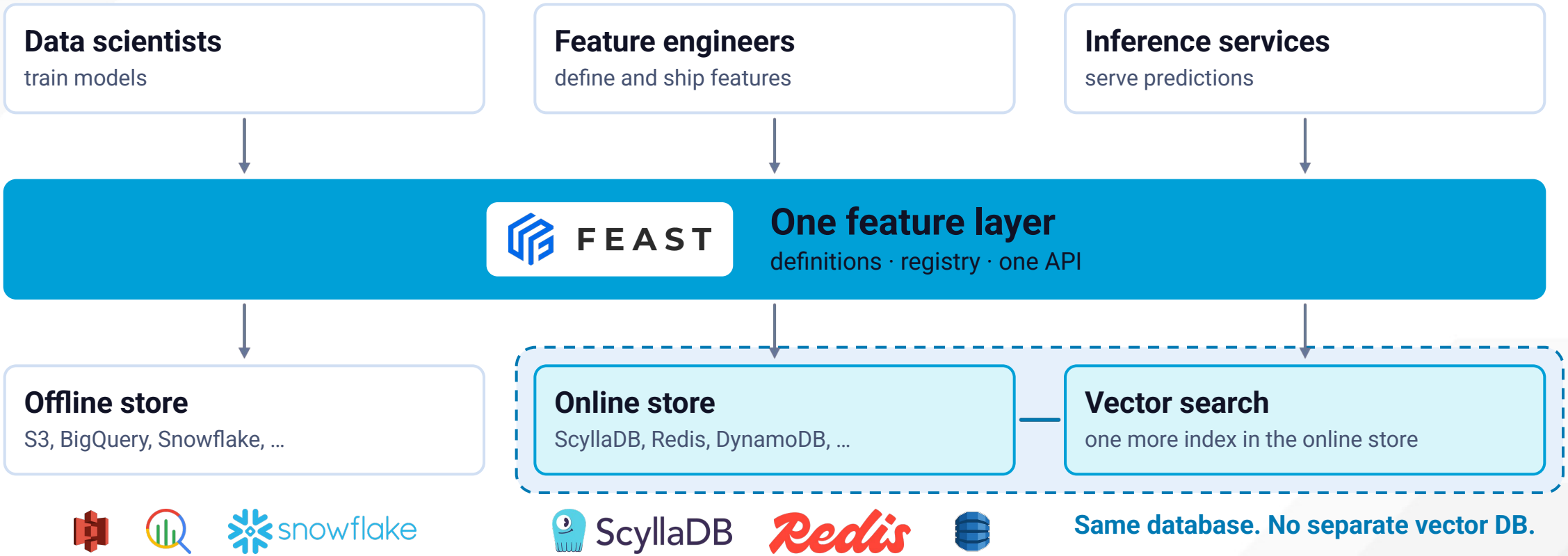
SCYLLA



Feast

Open source feature layer

One feature layer for every role



SCYLLA


use the
feature store's
built-in database
as online store
use whatever
database
I want as
online store
imgflip.com

SCYLLA

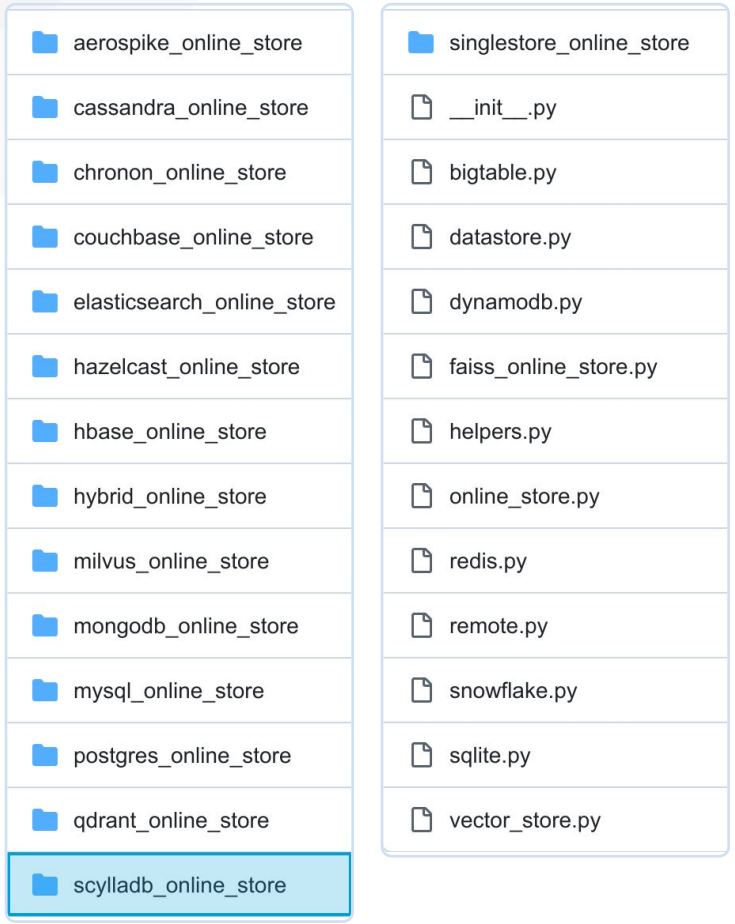
Can you bring your own online store?

Framework	Open source or managed	Online store(s)	Pluggable online store
Feast	Open source	Pluggable: ScyllaDB, Redis, DynamoDB, …	Yes
Featureform	Open source, managed	Pluggable	Yes
Feathr	Open source	Pluggable	Yes
Tecton	Managed	DynamoDB or Redis	Limited
Chalk	Managed	Redis, DynamoDB, Bigtable, Postgres	Limited
Databricks	Managed	Lakebase, DynamoDB, Aurora, RDS MySQL	Limited
Hopsworks	Open source, managed	RonDB (built-in)	No
Amazon SageMaker	Managed	Built-in tiers (InMemory on ElastiCache)	No
Vertex AI	Managed	Bigtable (built-in)	No

Sources: featurestore.org · docs.tecton.ai · docs.aws.amazon.com · docs.databricks.com · docs.cloud.google.com



Online store integrations in Feast



Aerospike | Apache Cassandra | Apache HBase | Bigtable

Couchbase | Datastore | DynamoDB | elasticsearch

Hazelcast | milvus | MongoDB | MySQL

PostgreSQL | Qdrant | Redis | ScyllaDB

SingleStore | snowflake | SQLite | also FAISS, Chronon

Source: github.com/feast-dev/feast/tree/master/sdk/python/feast/infra/online_stores

SCYLLA

Feast: online store config

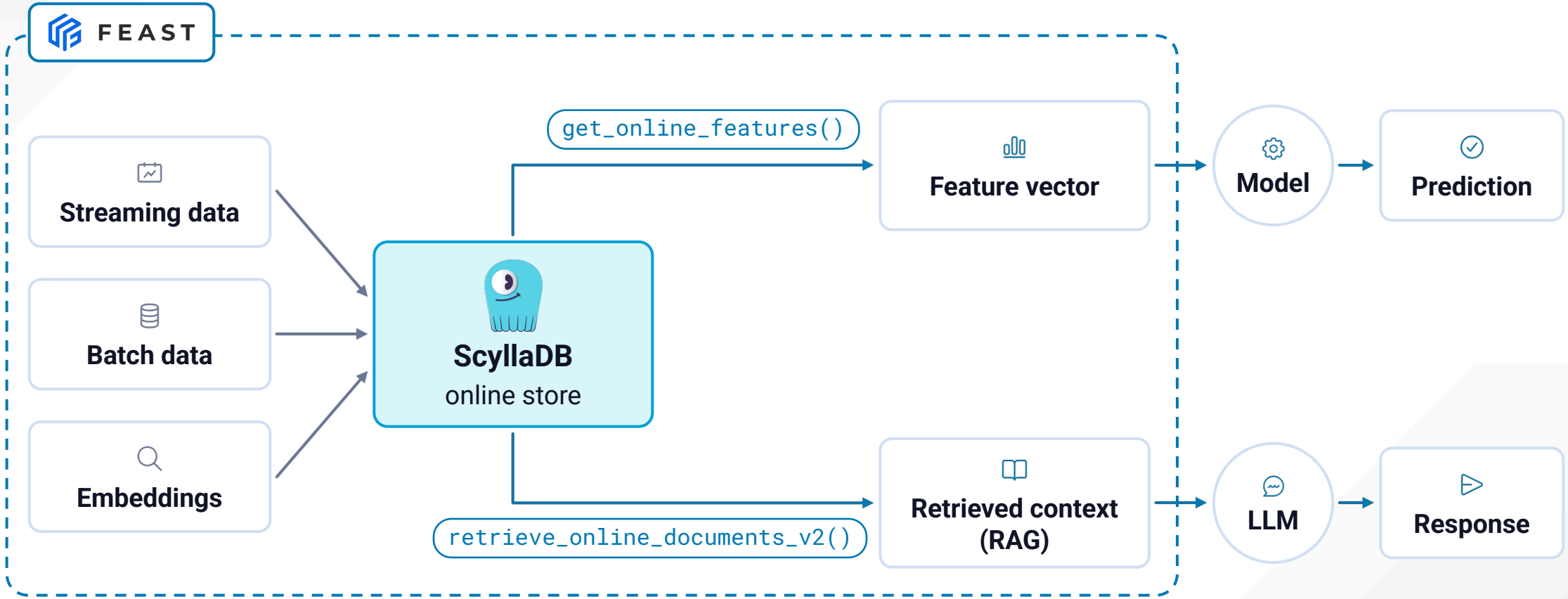
```
# pip install feast[scylladb]
project: my_feature_repo
registry: data/registry.db
provider: local
online_store:
  type: scylladb
  hosts:
    - node-0.xxxx.clusters.scylla.cloud
  keyspace: feast
  local_dc: AWS_US_EAST_1
```

```
from feast import FeatureStore

store = FeatureStore(repo_path=".")
features = store.get_online_features(
  features=[
    "driver_hourly_stats:conv_rate",
    "driver_hourly_stats:acc_rate",
  ],
  entity_rows=[{"driver_id": 1001}],
).to_dict()
```

feast apply creates the tables. feast materialize fills them. Shard-aware driver built in.

Features and vectors from one store



Based on: feast.dev/blog/scylladb-feast-online-store

SCYLLA

Define a vector field, then search it

```
Field(
  name="embedding",
  dtype=Array(Float32),
  tags={
    "vector_index": "true",
    "dimensions": "768",
    "similarity_function": "COSINE",
  },
)
```

```
result = store.retrieve_online_documents_v2(
  features=["documents:text",
            "documents:embedding"],
  query=[0.1, 0.2, ...],
  top_k=10,
  distance_metric="COSINE",
)
```

One feast apply creates the feature tables and the vector indexes.

Takeaways

1. Design tables for the query: one request reads one partition and few rows.
2. Cut cost with high node utilization, a TTL on every row and fewer data services.
3. Centralize access: one feature layer for training, serving and vectors.
4. Treat embeddings as features. Store them in the online store with a vector index. You do not need a separate vector database.
5. Keep the online store pluggable. If you have issues with the database, you can always switch without changing the feature code.

Thank you

attila.toth@scylladb.com · linkedin.com/in/attilapm